
Critical Review and Research Proposal

KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache

Prepared for Yandex Research ML Residency

Ivan A. Novosad
inovosad@hse.ru

1. Part I: Critical Review

1.1. Executive Summary

KIVI (Liu et al., 2024) proposes a tuning-free, asymmetric quantization scheme that compresses the key–value (KV) cache of large language models (LLMs) to 2 bits per element. The core contribution is an empirical observation that key and value activations exhibit distinct outlier distributions: keys contain persistent channel-wise outliers, whereas values do not. This motivates **per-channel quantization for keys** and **per-token quantization for values**, supplemented by a sliding-window residual buffer of $R \leq 128$ tokens in FP16. The authors report $2.6\times$ peak memory reduction on Llama-2-7B, support for $4\times$ larger batch sizes, and $2.35\text{--}3.47\times$ throughput gains on a single A100 GPU. KIVI requires no calibration data or fine-tuning and is compatible with weight-only quantization methods such as AWQ (Lin et al., 2024).

Despite the paper’s pioneering contribution to the quantized KV cache field, the authors mainly just observe and prove the claim from the weight-activation quant field: activations (in this case, keys) contain per-channel outliers. Retrospective analysis exposes weaknesses in its theoretical assumptions, experimental methodology, and evaluation scope. Later work has shown that KIVI’s post-RoPE quantization strategy is suboptimal and its baselines insufficient. I’ve tried to explicitly note when my criticisms rely on concurrent or even subsequent work, so we can fairly distinguish knowable limitations and insights that emerged only later.

1.2. Problem Relevance

The KV cache memory bottleneck is among the most consequential problems in LLM deployment. During autoregressive decoding, the KV cache grows linearly with both sequence length and batch size. For a 70B-parameter model serving 128K-token contexts, KV cache storage can exceed 160 GB of HBM, often surpassing the model weights them-

selves. This imposes hard ceilings on throughput, maximum context length, and cost-per-token in production systems. The problem’s severity has only increased since KIVI’s submission: context windows have expanded from 4K (Llama-2 era) to 128K+ (Llama-3, GPT-4 Turbo) and 1M+ (from Gemini 1.5 to Opus 4.6). KIVI targets this concrete problem: layer—activation compression on pre-trained models, without retraining needed.

By early 2025, the competitive set includes improved quantization methods (Hooper et al., 2025; Su et al., 2025; Zhang et al., 2024; Kang et al., 2024), token eviction (Zhang et al., 2023), hybrid approaches (Ramachandran et al., 2025), and architectural solutions like MLA or GQA. KIVI must be evaluated against this evolved landscape.

1.3. Strengths

1.3.1. CLEAR ASYMMETRIC INSIGHT WITH THOROUGH EMPIRICAL GROUNDING

The paper’s central observation – that keys and values require different quantization axes – is well supported by the activation distribution analysis (Figure 2 of Liu et al. (2024)) and by the quantization error statistics shown in Table 1. The table highlights that while key quantization benefits massively from a per-channel approach (reducing attention error from 47.00 to 9.60), values are relatively insensitive to the quantization granularity, justifying the mixed strategy.

The visualization of persistent channel-wise outliers in keys, contrasted with their absence in values, is compelling. The attention-sparsity argument for why value quantization error is attenuated (Section 3.2 of Liu et al. (2024), yielding an approximately $15\times$ error ratio in the attention output) provides an intuitive theoretical justification, though one with important caveats. Table 1 of the paper systematically ablates all four combinations of quantization axes (per-channel vs. per-token for **K** and **V**), confirming the asymmetric design.

Table 1. Relative error statistics averaged over all layers and all heads (reproduced from Table 2 of Liu et al. (2024)). Note the significant error reduction for Keys when switching to per-channel quantization, versus the stability of Values.

Llama-2-13B	K Per-Token	K Per-Channel
Avg. $\ \frac{\mathbf{X}_K - \mathbf{X}'_K}{\mathbf{X}_K} \ _F$	13.67	4.55
Avg. $\ \frac{\mathbf{A} - \mathbf{A}'}{\mathbf{A}} \ _F$	47.00	9.60
Attention sparsity	84.3%	
	V Per-Token	V Per-Channel
Avg. $\ \frac{\mathbf{X}_V - \mathbf{X}'_V}{\mathbf{X}_V} \ _F$	4.57	3.73
Avg. Δ	3.55	49.89

1.3.2. PLUG-AND-PLAY, TUNING-FREE DEPLOYMENT

Unlike KVQuant (Hooper et al., 2025) (which requires calibration data), GPTQ/AWQ (Lin et al., 2024) (which require weight calibration), or GQA/MQA (which require retraining), KIVI is fully post-training and requires zero additional data. This is a main practical advantage: deployment of KIVI on any supported model doesn’t require a calibration pipeline, significantly reducing integration difficulty.

1.3.3. ENGINEERING QUALITY AND EVALUATION BREADTH

The authors also address the practical latency bottlenecks by providing custom Triton kernels and fused operators. While I rely on the reported metrics to judge the efficiency of these kernels, the proposed data structure (streaming quantized cache with an FP16 buffer) is intuitively a strong solution for this specific hardware constraint. The transparency of the repository and the breadth of evaluation across different architectures (Llama-2, Falcon, Mistral) further strengthen the paper’s contribution.

1.4. Weaknesses

1.4.1. W1: FAILURE TO ACCOUNT FOR THE ROPE-QUANTIZATION INTERACTION

KIVI quantizes keys *after* applying Rotary Positional Embeddings (RoPE) (Su et al., 2024), yet its design premise – persistent channel-wise outliers – depends on the *pre-RoPE* distribution. RoPE rotates pairs of adjacent channels by a position-dependent angle θ_m . For a token at position m , an outlier originally confined to channel $2i$ is distributed across both channels $2i$ and $2i+1$ with magnitudes proportional to $\cos(\theta_m)$ and $\sin(\theta_m)$. This rotation weakens the channel-wise persistence that KIVI’s per-channel quantization relies upon.

In fairness to authors. This limitation was a concurrent discovery. KVQuant (Hooper et al., 2025) appeared on arXiv on January 31, 2024 – five days before KIVI’s February 5 submission – and the two papers were clearly developed independently. I couldn’t find any earlier papers that would connect RoPE to KV cache quantization difficulty. It would be unfair to characterize this as an oversight the KIVI authors should have caught. It is what it is, the RoPE interaction is a real limitation. KVQuant directly ablates the effect: pre-RoPE quantization yields a 0.82 perplexity-point improvement over post-RoPE on LLaMA-7B at 3-bit with per-channel granularity. They note that post-RoPE quantization is acceptable with per-token granularity (because rotation preserves token-vector norms), but KIVI uses per-channel for keys – precisely the case where post-RoPE is mathematically suboptimal. RotateKV (Su et al., 2025) later confirmed this, with pre-RoPE grouped-head rotation as one of its three components alongside outlier-aware channel reordering and attention-sink-aware quantization.

The KIVI paper doesn’t contain any mention of RoPE/Rotary/Positional in its methodology or ablation sections. It treats the key cache as a static activation tensor without addressing the dynamic geometric transformation that alters outlier distributions in all RoPE-based models (Llama, Mistral, Qwen). The omission was understandable given the concurrent discovery, but it leaves the paper’s core theoretical argument incomplete for the very models it evaluates.

1.4.2. W2: EFFECTIVE BIT-WIDTH SIGNIFICANTLY EXCEEDS THE ADVERTISED 2-BIT

KIVI claims “2-bit quantization,” but the FP16 residual buffer of $R = 128$ tokens inflates the effective bit-width substantially. The residual buffer is maintained per-head, per-layer (as I have found in the codebase, where residual length is applied uniformly to every attention head in every layer). The effective bits per element across a sequence of length L is:

$$b_{\text{eff}} = \frac{3(L - R)}{L} + \frac{16R}{L} = 3 + \frac{13R}{L}, \quad (1)$$

where the factor of 3 accounts for 2 data bits plus 1 bit of metadata overhead (one FP16 scale and one FP16 zero-point per group of $G = 32$ elements, yielding $(16+16)/32 = 1.0$ additional bit per quantized element). Table 2 lists the resulting effective bit-widths.

KIVI drops below 4 effective bits only at $L > 1,664$ tokens and *never* drops below 3.0 bits at any sequence length once metadata is properly accounted for. The “2-bit” label is asymptotically correct only if metadata overhead is ignored. I understand that the metadata overhead is inescapable, but KIVI’s report of $2.6\times$ peak memory reduction (versus the theoretical $8\times$ from pure 2-bit) looks suboptimal.

Table 2. Effective bit-width of KIVI with $R = 128$ and $G = 32$, including quantization metadata overhead.

Sequence Length	Effective Bits
$L = 512$	6.25
$L = 1,024$	4.63
$L = 2,048$	3.81
$L = 4,096$	3.41
$L = 32,768$	3.05
$L \rightarrow \infty$	3.00 (asymptote)

The FP16 residual buffer is near-universal – CQ, GEAR, and SKVQ all use comparable 128-token windows – so this critique applies broadly. However, KVQuant uses only the first token in FP16, and RotateKV uses targeted sinks with no large residual, explicitly criticizing large high-precision caches. KIVI never reports an effective bit-width metric, making its “2-bit” framing misleading at practical lengths. The paper’s own data (Table 3 of Liu et al. (2024)) shows that without the residual buffer, “fake” 2-bit quantization drops GSM8K accuracy on Llama-2-13B from 22.67% to 12.21% – a nearly 50% relative degradation. KIVI recovers to 20.77% only via the FP16 residual. The core 2-bit algorithm is fundamentally lossy; reported accuracy depends heavily on the high-precision buffer, particularly on short-to-moderate context benchmarks where the buffer constitutes a large fraction of the total cache.

1.4.3. W3: BASELINES ARE NEITHER FAIR NOR COMPREHENSIVE

KIVI compares exclusively against FP16 baselines and vanilla Round-to-Nearest (RTN) 4-bit per-token quantization – the weakest possible quantization baseline. By early 2024, RTN was already known to be insufficient for activation quantization due to outliers.

Missing quantization baselines. SmoothQuant (Xiao et al., 2024a), which introduced equivalent transforms for activation outliers, is cited but not compared against. ATOM (Zhao et al., 2024), which introduced mixed-precision for outlier channels and appeared on arXiv in October 2023, was available well before KIVI’s submission and is absent from the evaluation.

KVQuant appeared five days before KIVI, so missing it initially is fine. But it was still absent from the ICML camera-ready four months later.

Token eviction methods (Zhang et al., 2023; Xiao et al., 2024b) that compress the KV cache through a different mechanism are also absent, preventing assessment of where quantization vs. eviction lies on the Pareto frontier. While eviction and quantization are arguably orthogonal, compar-

ing them would have strengthened the paper

System baselines. Throughput claims of $2.35\text{--}3.47\times$ are measured against bare HuggingFace Transformers inference on a single A100 –not a production-grade serving system. By early 2024, vLLM with PagedAttention (Kwon et al., 2023) was the de facto standard, already achieving significant speedups through memory management alone. Without comparing “vLLM + KIVI” against “vLLM + PagedAttention + FlashAttention-2”, it is impossible to isolate gains from quantization vs. memory management.

The MLSys 2025 analysis by Gao et al. (2025) later demonstrated that KIVI’s throughput claims are framework-dependent: once FlashAttention and PagedAttention are enabled, KIVI shows no meaningful throughput improvement across KV lengths 1024–5120 ($0.98\times$ decode speedup at TP=1). Most strikingly, KIVI suffers out-of-memory errors at KV length 8192 with batch = 1 when the FP16 baseline does not, this is likely because quantization metadata and the residual window add overhead that exceeds savings at intermediate lengths.

Long-context gap. KIVI’s primary value proposition is long-context inference, yet its longest evaluation reaches only $\sim 30\text{K}$ tokens. LongBench tasks span 4K–8K tokens – nowhere near the regime where KV cache dominates memory. The single-passkey NIAH test at 20K words is nearly saturated and cannot reveal quantization-induced degradation. More demanding benchmarks (RULER at 128K, MRCR v2 (Vodrahalli et al., 2024)) did not exist at submission time, but even by early-2024 standards, testing only up to 30K for a memory-focused method was a gap.

1.4.4. W4: INSUFFICIENT REASONING EVALUATION WHERE ERRORS COMPOUND

GSM8K alone is insufficient for evaluating reasoning robustness – its problems require only 3–8 steps. This is **the fairest criticism knowable at submission time**: GSM8K (2021) and MATH (2021) were standard benchmarks well before KIVI’s February 2024 submission, and omitting MATH-level reasoning evaluation was a genuine gap.

PM-KVQ (Liu et al., 2025) provides later evidence: on DeepSeek-R1-Distill-Qwen-7B, KIVI-2bit drops AIME-2024 accuracy from 41.04% to 32.08% (21.8% relative degradation) and LiveCodeBench from 26.29% to 19.00% (27.7% relative degradation). These reasoning models generate thousands of intermediate tokens, filling the KV cache with intermediate reasoning steps that must be accessed with high precision. KIVI’s uniform 2-bit quantization introduces noise across the entire chain, and the residual buffer ($R = 128$) covers only the most recent tokens.

A note on error compounding. Quantization errors in the KV cache are often described as “compounding,” but the picture seems more nuanced. The errors ΔK and ΔV are fixed perturbations – each cached entry is written once, so they should not grow over time on their own. What likely compounds instead is the autoregressive pathway: if quantization noise at step t nudges the model toward a wrong token, that token enters the context and can corrupt subsequent steps. This is similar to the exposure-bias model, where the probability of a fully correct n -step sequence decays roughly as $(1 - \epsilon)^n$, though in practice it seems that only about 5–10% of tokens are “key tokens” where errors actually cascade. KIVI does not appear to discuss this mechanism either.

The LiveCodeBench degradation is partly attributed to structured-output failures: quantization noise causes token-level errors (e.g., predicting “(” instead of “{”), breaking JSON/code validity even when the underlying logic is partially correct. No code generation benchmarks (HumanEval, MBPP) are included in the original evaluation.

1.4.5. W5: LIMITED EVALUATION ON GQA AND MULTIMODAL MODELS

Llama-3 results appear only in an appendix with limited testing. Because GQA shares 8 KV heads across 32 query heads, a badly quantized channel creates **correlated noise** across all 4 associated query heads –reducing noise diversity compared to MHA, where each head has independently quantized KV pairs. The appendix data shows larger accuracy drops than on Llama-2: LCC falls from 57.00 to 50.84, RepoBench-P from 51.22 to 46.65. Mistral-7B also used GQA and existed before submission; testing it without analyzing GQA implications was a knowable gap, in my opinion. RotateKV’s multimodal evaluation later showed KIVI-2bit scoring 27.8 vs. FP16’s 52.0 across 8 tasks –though multimodal benchmarking was not established practice in early 2024.

1.5. Additional Technical Concerns

KIVI uses the simplest possible quantization method (Round-to-Nearest with min/max scaling) without exploring non-uniform grids, L_p -norm optimization, or learned scaling. For key caches with known channel-wise outliers, even basic calibration-based scaling would improve grid efficiency. On the systems side, KIVI’s per-channel layout assumptions conflict with vLLM’s non-contiguous PageAttention, and as of early 2026, I found no major serving framework (vLLM, TensorRT-LLM) has adopted KIVI’s approach.

The MLSys 2025 analysis (Gao et al., 2025) also identified a “verbose-response” phenomenon where all tested KV compression methods cause $1.55\text{--}1.76\times$ longer out-

puts, potentially erasing throughput gains when measured end-to-end.

Table 3 summarizes KIVI’s positioning against state-of-the-art (2024-2025) KV cache compression methods.

1.6. Practical Feasibility Assessment

KIVI still makes sense in a narrow lane: 4-bit on consumer GPUs with moderate context lengths and latency tolerance. Its tuning-free nature is a genuine advantage for rapid prototyping.

However, its effectiveness degrades substantially under the following conditions:

1. **2-bit precision on RoPE-based architectures** (all modern LLMs), where the post-RoPE per-channel approach introduces avoidable error.
2. **Long-context tasks** where retrieval targets fall outside the residual buffer.
3. **Multi-step reasoning or code generation**, where quantization errors cascade through the autoregressive process when corrupted outputs enter the context.
4. **GQA architectures**, where quantization noise creates correlated errors across grouped query heads.
5. **Production serving**, where no major framework (vLLM, TensorRT-LLM) has adopted KIVI, and the MLSys 2025 analysis (Gao et al., 2025) shows its throughput advantages vanish under production-grade serving stacks.

For enterprise-grade serving, long-context retrieval (128K+), or reasoning-intensive applications, practitioners should adopt RoPE-aware methods (Hooper et al., 2025; Su et al., 2025) or adaptive mixed-precision frameworks (Liu et al., 2025; Ramachandran et al., 2025). For hardware with native FP8 support (H100, Blackwell), FP8 KV cache via TensorRT-LLM offers a superior throughput/accuracy trade-off with zero dequantization overhead.

2. Part II: Research Proposal

*RoPE-Aware Adaptive KV Cache Quantization with
Semantic Sensitivity
for Long-Context Reasoning Models*

2.1. Problem Statement

Three problems now compound: contexts have grown to 128K+ (even 1M+ Gemini 1.5 and Opus 4.6), reasoning models generate thousands of intermediate tokens

Table 3. KIVI vs. state-of-the-art KV cache compression methods on Llama-2-13B GSM8K (8-shot CoT). Panel A shows results from RotateKV’s unified evaluation (Su et al., 2025) (Table 5), where all methods are tested under identical conditions. Panel B shows KIVI’s self-reported numbers (Liu et al., 2024) (Table 3), which use a different evaluation setup and yield a lower FP16 baseline (22.67% vs. 25.40%). Relative degradation from the respective FP16 baseline is more informative than cross-panel absolute comparisons.

	Method	Avg. Bits	Pre-RoPE?	GSM8K (%)	Tuning-Free?	Key Innovation
Panel A	FP16	16	—	25.40	—	Baseline
	RotateKV	2.25	Yes	25.09	No	Outlier-aware rotation
	ZipCache	3.23	No	25.02	No	Salient token identification
	KIVI	2.50	No	24.64	Yes	Asymmetric K/V axes
	GEAR	4.03	No	22.74	Yes	Low-rank + sparse residual
	MiKV (Yang et al., 2024)	3.23	No	21.00	No	Mixed-precision per-layer
Panel B	FP16	16	—	22.67	—	Baseline
	KIVI-4	~4	No	23.65	Yes	Asymmetric K/V axes
	KIVI-2	~2	No	20.77	Yes	Asymmetric K/V axes

Note: FP16 baselines differ across evaluation setups (check the sources). Panel A numbers are directly comparable to each other; Panel B numbers are directly comparable to each other. Cross-panel comparison requires caution: KIVI’s self-reported 20.77% (Panel B, baseline 22.67%) and RotateKV’s evaluation of KIVI at 24.64% (Panel A, baseline 25.40%) yield similar relative degradation (~8–3%). KVQuant, CQ, PM-KVQ, and QAA do not report GSM8K on Llama-2-13B and are therefore omitted.

(DeepSeek-R1, QwQ, o1-style models), and 2-bit compression still breaks on both. Current methods solve these partially:

- **KVQuant / RotateKV** (Hooper et al., 2025; Su et al., 2025) solve the RoPE interaction problem and achieve excellent perplexity, but apply uniform bit-width across the entire sequence. They do not differentiate between semantically critical tokens and transitional filler.
- **PM-KVQ / ThinKV** (Liu et al., 2025; Ramachandran et al., 2025) introduce mixed-precision or semantic-aware eviction for reasoning models, but PM-KVQ uses a fixed progressive schedule (not input-dependent), and ThinKV’s token classification relies on heuristics that may not generalize across model families.
- **QAA** (Dong et al., 2024) employs attention-aware dynamic quantization with a “persistence of importance” hypothesis, approximating future attention from a window of past attention scores. I compare against QAA directly in Section 2.2.3.

Research question. Can a KV cache compression method be designed that (a) correctly handles RoPE via pre-quantization outlier smoothing, (b) dynamically allocates bit-width based on each token’s measured importance to ongoing computation, and (c) maintains <10% degradation on both long-context retrieval (128K, multi-needle) and multi-step reasoning (AIME, code generation), at an effective compression ratio of ~2.7 bits per element – tighter than KIVI’s 3.0-bit asymptote (Table 4) and competitive with

RotateKV’s 2.25 average bits, while retaining the ability to preserve critical tokens at full precision?

2.2. Proposed Method: SmoothQ-Adapt

SmoothQ-Adapt compresses the KV cache in three stages: outlier smoothing before RoPE, L_p -norm quantization, and attention-driven bit allocation. The target is RoPE-based models (Llama-3, Qwen-2.5, DeepSeek-R1-Distill) at long contexts.

The three stages are not independent modules – they interact through the attention score distribution. Pre-RoPE smoothing (Stage 1) reduces the variance of key magnitudes across channels, which in turn reduces the variance of $\mathbf{QK}^\top$ dot products and produces *sharper, more stable* softmax attention distributions. With less attention score variance, the EMA importance signal in Stage 3 fluctuates less between recomputation windows, reducing spurious tier promotions and demotions – the primary failure mode of dynamic allocation over long contexts.

A rough sketch of the effect size: if pre-RoPE smoothing halves the key magnitude variance (plausible given KVQuant’s 145% MSE reduction from post- to pre-RoPE quantization), the variance of the $\mathbf{QK}^\top$ logits drops proportionally, since $\text{Var}(\mathbf{q}^\top \mathbf{k}) \propto \text{Var}(\mathbf{k})$ for fixed queries. Lower logit variance sharpens the softmax: the entropy of a softmax distribution decreases as its input variance decreases relative to the temperature $1/\sqrt{d}$. Concretely, if quantization noise adds σ_{noise}^2 to logit variance, the noise-to-signal ratio $\sigma_{\text{noise}}^2/\sigma_{\text{logit}}^2$ doubles when σ_{logit}^2 is halved – but the absolute noise also drops (because the keys being

quantized have smaller range), and the net effect depends on which reduction dominates. I do not claim this guarantees the interaction; I claim it makes the effect plausible enough to warrant the experimental test in RQ3.

RQ3 tests this by measuring attention entropy and tier-transition frequency with and without the Stage 1 substrate.

2.2.1. STAGE 1: PRE-ROPE OUTLIER SMOOTHING VIA LEARNED CHANNEL SCALING

Motivation. KVQuant (Hooper et al., 2025) demonstrated that pre-RoPE quantization is strictly superior for per-channel methods. RotateKV (Su et al., 2025) showed that Hadamard rotation can eliminate outliers. I combine both insights: add a learned per-channel scaling step before RoPE.

Method. Using a small calibration set (256 sequences from RedPajama, following KVQuant’s protocol), I learn per-layer, per-channel scaling vectors $\mathbf{s}_k$ and $\mathbf{s}_v$ for keys and values, respectively. The scaling factors are computed to minimize the L_p -norm ($p = 0.7$, following HQQ’s insight (Badri & Shaji, 2023) that $p < 1$ is more robust to heavy-tailed distributions) of the quantization error on the calibration set. During inference, keys are scaled by $\mathbf{s}_k$ before RoPE application, and the inverse scaling $\mathbf{s}_k^{-1}$ is absorbed into the query projection weights ($\mathbf{Q}' = \mathbf{Q} \cdot \text{diag}(\mathbf{s}_k^{-1})$), incurring zero runtime cost. For values, $\mathbf{s}_v$ is applied before quantization and its inverse during dequantization.

This is cheaper than RotateKV’s dense Hadamard rotation (diagonal scaling vs. matrix multiply) and, unlike SmoothQuant/AWQ, targets the pre-RoPE key distribution specifically.

Addressing the scaling-alone failure mode. RotateKV (Su et al., 2025) demonstrated that channel-wise scaling alone fails catastrophically at 2-bit (PPL 16.97 vs. 8.05 for Hadamard rotation on LLaMA-2-7B, their Table 1). This result poses a direct challenge to Stage 1. I hypothesize that the failure stems from a specific interaction: heuristic scaling (SmoothQuant-style diagonal Λ) redistributes outlier magnitude but does not reduce the *within-group variance* that RTN’s min/max grid must accommodate.

Stage 2’s L_p solver directly targets this residual: by optimizing the quantization grid under a heavy-tailed loss rather than min/max, it can absorb the variance that scaling alone leaves behind. Scaling reduces the inter-channel dynamic range problem; L_p quantization reduces the intra-group tail sensitivity. Neither suffices alone, but they target complementary failure modes. A second difference: the scaling vectors are learned to minimize L_p reconstruction error on the calibration set, whereas RotateKV’s “Smooth-Only” baseline uses activation-magnitude heuristics without

joint optimization. RQ1 is designed to test this hypothesis directly: if learned scaling + L_p quantization matches or approaches rotation-based methods at 2-bit, the hypothesis is confirmed; if it falls short, I will characterize which failure mode persists and whether rotation remains necessary. The goal is not to replace rotation but to test whether learned scaling + robust quantization closes the gap enough to justify its lower computational cost.

If learned scaling + L_p quantization still falls materially short of rotation at 2-bit, the fallback is straightforward: replace Stage 1 with RotateKV’s Hadamard rotation as the pre-RoPE substrate and test whether Stages 2 and 3 still add value on top of the strongest known base. This fallback isolates the contribution of L_p quantization and dynamic allocation independent of the smoothing strategy.

2.2.2. STAGE 2: ROBUST PER-GROUP QUANTIZATION WITH L_p -NORM OPTIMIZATION

Method. Even after outlier smoothing, within-group variation causes RTN’s min/max grids to overfit extreme values. For each quantization group of $G = 32$ elements, replace min/max scaling with a half-quadratic L_p minimization ($p = 0.7$):

$$\min_s \|\mathbf{x} - Q(\mathbf{x}, s)\|_p^p, \quad (2)$$

where $Q(\mathbf{x}, s)$ denotes quantization with scale s . The sub-quadratic exponent penalizes large errors less aggressively than L_2 , producing grids that better accommodate residual outliers. At $G = 32$, the solver converges in 3–5 iterations with negligible latency overhead.

The L_p solver follows HQQ (Badri & Shaji, 2023), originally developed for weight quantization; I integrate it into the pre-RoPE pipeline and study how it interacts with KV cache activations. I benchmark against HQQ’s existing KV cache mode as a baseline.

2.2.3. STAGE 3: ATTENTION-DRIVEN DYNAMIC BIT-WIDTH ALLOCATION

Motivation. Uniform 2-bit quantization destroys reasoning chains (KIVI loses 21.8% on AIME). Fixed progressive schedules (PM-KVQ (Liu et al., 2025)) cannot adapt to input-dependent importance patterns. Attention scores are already computed during inference and directly signal token importance.

Method. The method maintains a per-token importance score $I(t)$, updated as an EMA of the attention mass received across all heads:

$$I(t) \leftarrow \alpha \cdot I(t) + (1 - \alpha) \sum_h a_h(t). \quad (3)$$

Tokens are assigned to three tiers: **Tier A (FP16)** for attention sinks, recency-window tokens, and the top 5% by $I(t)$; **Tier B (4-bit)** for the 75th–95th percentile; **Tier C (2-bit)** for the rest.

Tier boundaries are recomputed every $K=256$ steps. Promotion uses a stored low-rank quantization residual following GEAR’s approach (Kang et al., 2024); demotion simply quantizes the token’s entries to the lower precision. EMA has an obvious weakness: it biases toward recently important tokens. A variable binding from step 50 could matter again at step 2000, long after its score has decayed. To mitigate this, tokens that ever reached Tier A retain a minimum importance floor preventing demotion below Tier B, and the GEAR-style low-rank residual enables partial reconstruction if re-promotion is needed.

Tier A is functionally a residual buffer, like KIVI’s fixed FP16 window (Section 1.4.2), and I should be explicit about its composition: it has a *fixed* component (4 sink tokens + 64 recency tokens, always FP16 regardless of importance) and a *dynamic* component (the top 5% by EMA score). The fixed part exists for the same reason KIVI’s residual does – recent tokens and attention sinks are reliably important. The difference is that KIVI allocates all 128 FP16 slots to recency, while SmoothQ-Adapt reserves most of its FP16 budget for tokens selected by measured importance, which matters when critical tokens are old and non-recent (e.g., variable bindings from early in a reasoning chain).

Effective bit-width accounting. Part I (Section 1.4.2) criticized KIVI for omitting this metric. The same accounting applied to SmoothQ-Adapt: For a sequence of length L with Tier A fraction f_A , Tier B fraction f_B , and Tier C fraction $f_C = 1 - f_A - f_B$, the effective bits per KV element are:

$$b_{\text{eff}} = 16 f_A + (4 + m_4) f_B + (2 + m_2) f_C + o_{\text{EMA}} + o_{\text{LR}}, \quad (4)$$

where m_b is the per-element metadata overhead at b -bit (scale + zero-point per group of $G=32$: $m_b = 32/32 = 1.0$ bit for both tiers), $o_{\text{EMA}} = 16/L$ accounts for one FP16 importance score stored per token amortized over elements ($d=128$ channels, so $16/(128) \approx 0.125$ bits per element), and o_{LR} is the GEAR-style low-rank residual overhead for Tier B/C tokens (rank $r=5$, giving $(n+d) \cdot r \cdot 16/(n \cdot d) \approx 0.15$ bits per element at $L=4096$).

Table 4 instantiates this for default configuration at a 2.5-bit budget ($f_A=0.05$, $f_B=0.20$, $f_C=0.75$) across sequence lengths, including 4 attention-sink tokens and a 64-token recency window in Tier A.

SmoothQ-Adapt converges to approximately **2.70 effective bits**—0.30 bits lower than KIVI’s 3.00-bit asymptote, because KIVI’s 128-token FP16 residual is a fixed overhead whereas the Tier A fraction shrinks proportionally with L . At short sequences ($L < 1024$), both methods incur high

Table 4. Effective bit-width of SmoothQ-Adapt at 2.5-bit budget target. Tier A includes top-5% by importance + 4 sinks + 64 recency tokens. At short sequences, the fixed sink/recency allocation inflates Tier A beyond 5%.

Seq. Length	f_A	f_B	f_C	b_{eff}	vs. KIVI
$L = 512$	0.18	0.16	0.66	4.98	6.25
$L = 2,048$	0.08	0.18	0.74	3.33	3.81
$L = 4,096$	0.07	0.19	0.74	3.06	3.41
$L = 32,768$	0.05	0.20	0.75	2.73	3.05
$L = 128,000$	0.05	0.20	0.75	2.71	3.01
$L \rightarrow \infty$	0.05	0.20	0.75	2.70	3.00

effective bit-widths; the gap widens at longer sequences, which is where KV cache compression is most needed.

Note that at $L=512$, SmoothQ-Adapt’s 4.98 bits appears lower than KIVI’s 6.25, but the comparison is not direct: KIVI’s overhead comes from a contiguous FP16 residual, while mine comes from importance-ranked Tier A tokens and fixed sinks/recency slots, which have different semantics and overhead profiles. The meaningful comparison is at $L \geq 4096$, where both methods’ fixed-overhead components are amortized. I report this metric for all experiments.

Relation to QAQ. The closest prior method is QAQ (Dong et al., 2024), which uses windowed past-attention for importance prediction. I replace its short window with EMA streaming (more stable over long contexts), use low-rank residuals instead of full-precision backup copies (Kang et al., 2024), and benefit from the improved quantization substrate of Stages 1–2.

Memory budget constraint. The method enforces a global memory budget B (e.g., equivalent to uniform 2.5 bits per token) by adjusting the percentile thresholds. If the current allocation exceeds B , the importance threshold for Tier A is raised, demoting the least-important high-precision tokens. The budget cap guarantees a fixed memory footprint regardless of tier distribution.

2.2.4. SYSTEM DESIGN: FUSED KERNEL ARCHITECTURE

To avoid the dequantization-overhead trap identified by Gao et al. (2025), all three stages are fused into efficient kernels: (1) a Triton kernel combining pre-RoPE scaling, RoPE application, and L_p -quantization in a single pass, (2) a tier-aware attention kernel dispatching to different dequantization paths based on each token’s bit-width, and (3) integration with vLLM’s PagedAttention (Kwon et al., 2023) via tier-tagged page headers enabling mixed-precision pages.

SmoothQ-Adapt assumes RoPE-based attention and moderate attention sparsity. It does not apply to ALiBi or absolute-position architectures. Under very tight budgets (below 2 ef-

fective bits), Tier A cannot retain enough tokens and quality will degrade. Dense-attention workloads like full-document summarization, where attention is spread uniformly, will see less benefit from dynamic allocation since fewer tokens can be safely compressed to 2-bit.

2.3. Experimental Plan

2.3.1. MODELS

Primary: Llama-3-8B-Instruct (GQA, 128K context), Llama-3-70B-Instruct (scaling behavior), Qwen-2.5-7B-Instruct (alternative RoPE implementation). *Reasoning:* DeepSeek-R1-Distill-Qwen-7B (long-CoT), DeepSeek-R1-Distill-Llama-8B. *Multimodal:* LLaVA-v1.5-7B (robustness to non-text activation distributions).

2.3.2. BASELINES

The core comparison set is FP16 (upper bound), KIVI-2bit (Liu et al., 2024) (the method I build on), RotateKV-2bit (Su et al., 2025) (best RoPE-aware method), QAA (Dong et al., 2024) (closest prior art for dynamic allocation), and vLLM + FP8 (hardware baseline on H100). I additionally compare against KVQuant (Hooper et al., 2025), PM-KVQ (Liu et al., 2025), and H2O (Zhang et al., 2023) as resources permit. All methods are evaluated at matched effective memory budgets. An internal ablation – Stages 1+2 only at uniform 3-bit (no dynamic allocation, matched to SmoothQ-Adapt’s 2.7-bit budget) – is included as a core baseline across all RQs to isolate Stage 3’s contribution.

2.3.3. BENCHMARKS AND RESEARCH QUESTIONS

RQ1: Does pre-RoPE smoothing with learned scaling improve upon Hadamard rotation? Ablation on WikiText-2/C4 perplexity at 2-bit (Llama-3-8B, Qwen-2.5-7B), comparing KIVI post-RoPE, QuaRot-style Hadamard, RotateKV’s rotation, and the learned scaling. I expect comparable accuracy to RotateKV at lower computational cost, because diagonal scaling is cheaper than dense rotation and allows per-layer tuning.

RQ2: Does L_p -norm quantization improve over RTN at 2-bit? WikiText-2 perplexity and GSM8K accuracy comparing RTN, MSE-optimal, HQQ’s L_p (Badri & Shaji, 2023), and my L_p ($p=0.7$) within the pre-RoPE pipeline. Expected gain: 0.2–0.5 PPL. Since HQQ tuned $p = 0.7$ for weight distributions, which differ from KV cache activations, I would also tune this $p \in [0.5, 1]$ to find the right operating point.

RQ3: Does dynamic bit-width allocation preserve reasoning chain integrity? *Benchmarks:* AIME-2024 (Pass@1), LiveCodeBench (Pass@1), MATH-500, Hu-

manEval on DeepSeek-R1-Distill-Qwen-7B. I compare uniform 2-bit (KIVI), uniform 2-bit with pre-RoPE (RotateKV), QAA (dynamic allocation without pre-RoPE), PM-KVQ mixed-precision, and SmoothQ-Adapt at matched memory budget (2.5 effective bits).

I expect that dynamic allocation will maintain >90% of FP16 reasoning accuracy at ~2.7 effective bits, narrowing the gap between KIVI’s ~78% (at 3.0 effective bits) and PM-KVQ’s ~98% (at higher effective bit-width with progressive scheduling). Matching PM-KVQ at lower bit-width is unlikely at 2.7 bits; >90% is the realistic target given that proposed method lacks PM-KVQ’s advantage of delaying all quantization until memory pressure forces it. Comparing against QAA isolates the EMA vs. windowed-attention question. Comparing against RotateKV isolates the pre-RoPE substrate. I also analyze which tokens are promoted to high-precision tiers during reasoning – I expect intermediate calculation results and variable bindings to receive high attention and thus high precision.

In addition to independent testing of each stage, I want to test whether first two stages affect the stage 3’s behavior. If better quantization substrates change the very tokens which cross the importance thresholds, that would prove the interaction effect of the stages.

To isolate Stages 1+2 only (pre-RoPE smoothing + L_p quantization at uniform 2-bit, no dynamic allocation), tested at matched effective bit-width by adjusting the uniform bit-width upward to 3-bit to match SmoothQ-Adapt’s 2.7-bit budget.

RQ4: Does the method maintain fidelity at 128K context with adversarial retrieval? *Benchmarks:* RULER (Hsieh et al., 2024) (QA tasks sourced from SQuAD and HotPotQA, NIAH variants) at 32K, 64K, and 128K; MRCR v2 (Vodrahalli et al., 2024) with multiple adversarial needles at 128K. MRCR v2 requires distinguishing between multiple semantically similar cached passages, and quantization noise can cause “collision” between similar vectors, leading to coreference errors. I expect needle tokens to land in Tier A (they receive high attention at retrieval time), leaving filler in Tier C. I report accuracy as a function of needle depth (position within the context) to characterize the method’s robustness profile.

RQ5: What is the end-to-end system throughput? vLLM serving on $1 \times A100-80GB$ and $1 \times H100-80GB$; batch sizes 1/8/32/64; context lengths 4K/32K/128K. Metrics: tokens-per-second, time-to-first-token, peak memory. Baselines: vLLM + FP16 + Flash Attention (one of its versions), vLLM + FP8 (H100 native). I decompose latency into prefill, quantization overhead, and decoding, and report end-to-end numbers inclusive of any verbose-response

effects (Gao et al., 2025).

I do not propose a fix for the verbose-response phenomenon itself – this appears to be a fundamental property of lossy KV compression, not specific to any method. However, if SmoothQ-Adapt’s dynamic allocation preserves higher fidelity for high-attention tokens, the verbosity effect may be smaller than for uniform quantization methods. RQ5 tests this by comparing output length distributions across methods at matched memory budgets.

RQ6: Cross-architecture generalization. I test scaling (Llama-3-8B → 70B), multimodal robustness (LLaVA-v1.5-7B), and sliding-window attention (Mistral-7B), specifically checking whether GQA correlated-noise effects (Section 1.4.5) are mitigated.

2.4. Expected Contributions

The main contribution is combining pre-RoPE smoothing, L_p quantization, and dynamic bit allocation into a single pipeline. I pair this with the first KV cache compression evaluation on MRCR v2 (128K, multi-needle), AIME-2024, and LiveCodeBench at matched memory budgets, a production-ready vLLM integration with fused kernels, and an analysis of which tokens get promoted during reasoning chains.

2.5. Estimated Timeline

Table 5. Estimated research timeline.

Phase	Activities
Weeks 1–3	Reproduce baselines; implement pre-RoPE scaling
Weeks 4–6	L_p quantization; EMA importance scoring; tier management
Weeks 7–9	Fused Triton kernels; vLLM integration
Weeks 10–12	Full evaluation (RQ1–RQ6); ablations; writing

Weeks 7–9 carry the most risk: writing fused Triton kernels for mixed-precision PagedAttention is the critical path, and I have limited prior kernel development experience. If fused kernels are not ready by Week 9, I fall back to unfused dequantization to validate the algorithm and report kernel-free throughput numbers for RQ5, deferring kernel optimization to follow-up work.

References

Badri, H. and Shaji, A. Half-quadratic quantization (HQQ) of large machine learning models. *arXiv preprint*, 2023. URL https://mobiusml.github.io/hqq_blog/.

Dong, S., Cheng, W., Qin, J., and Wang, W. QAQ: Quality

adaptive quantization for LLM KV cache. 2024. URL <https://arxiv.org/abs/2403.04643>.

Gao, W., Zhou, X., Sun, P., Zhang, T., and Wen, Y. Rethinking key-value cache compression techniques for large language model serving. 2025. URL <https://arxiv.org/abs/2503.24000>. MLSys 2025.

Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., and Gholami, A. Kvquant: Towards 10 million context length llm inference with kv cache quantization. 2025. URL <https://arxiv.org/abs/2401.18079>.

Hsieh, C.-P., Sun, S., Krizan, S., Acharya, S., Rekesh, D., Jia, F., Zhang, Y., and Ginsburg, B. RULER: What’s the real context size of your long-context language models? 2024. URL <https://arxiv.org/abs/2404.06654>.

Kang, H., Zhang, Q., Kundu, S., Jeong, G., Liu, Z., Krishna, T., and Zhao, T. GEAR: An efficient KV cache compression recipe for near-lossless generative inference of LLM. 2024. URL <https://arxiv.org/abs/2403.05527>.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. 2023. URL <https://arxiv.org/abs/2309.06180>.

Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S. Awq: Activation-aware weight quantization for llm compression and acceleration. 2024. URL <https://arxiv.org/abs/2306.00978>.

Liu, T., Li, S., Yang, J., Zhao, T., Zhou, F., Song, X., Dai, G., Yan, S., Yang, H., and Wang, Y. Pm-kvq: Progressive mixed-precision kv cache quantization for long-cot llms. 2025. URL <https://arxiv.org/abs/2505.18610>.

Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Ramachandran, A., Neseem, M., Sakr, C., Venkatesan, R., Khailany, B., and Krishna, T. Thinkv: Thought-adaptive kv cache compression for efficient reasoning models. 2025. URL <https://arxiv.org/abs/2510.01290>.

- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. ISSN 0925-2312. doi: <https://doi.org/10.1016/j.neucom.2023.127063>. URL <https://www.sciencedirect.com/science/article/pii/S0925231223011864>.
- Su, Z., Chen, Z., Shen, W., Wei, H., Li, L., Yu, H., and Yuan, K. Rotatekv: Accurate and robust 2-bit kv cache quantization for llms via outlier-aware adaptive rotations. 2025. URL <https://arxiv.org/abs/2501.16383>.
- Vodrahalli, K., Ontanon, S., Tripuraneni, N., Xu, K., Jain, S., Shivanna, R., Hui, J., Dikkala, N., Kazemi, M., Fatemi, B., Anil, R., Dyer, E., Shakeri, S., Viji, R., Mehta, H., Ramasesh, V., Le, Q., Chi, E., Lu, Y., Firat, O., Lazaridou, A., Lespiau, J.-B., Attaluri, N., and Olszewska, K. Michelangelo: Long context evaluations beyond haystacks via latent structure queries. 2024. URL <https://arxiv.org/abs/2409.12640>.
- Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., and Han, S. Smoothquant: Accurate and efficient post-training quantization for large language models. 2024a. URL <https://arxiv.org/abs/2211.10438>.
- Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. 2024b. URL <https://arxiv.org/abs/2309.17453>.
- Yang, J. Y., Kim, B., Bae, J., Kwon, B., Park, G., Yang, E., Kwon, S. J., and Lee, D. No token left behind: Reliable KV cache compression via importance-aware mixed precision quantization. 2024. URL <https://arxiv.org/abs/2402.18096>.
- Zhang, T., Yi, J., Xu, Z., and Shrivastava, A. Kv cache is 1 bit per channel: Efficient large language model inference with coupled quantization. 2024. URL <https://arxiv.org/abs/2405.03917>.
- Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., Wang, Z., and Chen, B. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL <https://arxiv.org/abs/2306.14048>.
- Zhao, Y., Lin, C.-Y., Zhu, K., Ye, Z., Chen, L., Zheng, S., Ceze, L., Krishnamurthy, A., Chen, T., and Kasikci, B. Atom: Low-bit quantization for efficient and accurate llm serving. 2024. URL <https://arxiv.org/abs/2310.19102>.